Midterm Progress Report

Project: Building Reasoning LLMs from Scratch using Chain-of-Thought Reasoning (DeepSeek R1)

WIDS'25 - Analytics Club, IIT Bombay

Name: Meet Agrawal

Contents

1 Introduction

This report documents the progress made during the first two weeks of the Winter in Data Science (WIDS) 2025 project. The project's objective is to deconstruct and implement "Reasoning" Large Language Models (LLMs), drawing inspiration from architectures like DeepSeek R1. Key topics covered include Chain-of-Thought (CoT) reasoning, Verifier models, Markov Decision Processes (MDPs), etc.

Artificial Intelligence is shifting from fast-pattern recognition toward slower-reasoning. LLM models like GPT-3 are strong at predicting text, they still find multi-step logical and mathematical reasoning challenging.

This project focuses on overcoming these limitations by combining Reinforcement Learning (RL) with Chain-of-Thought (CoT) prompting. The goal is to train models that can reason step by step, check their own logic, and correct mistakes along the way.

2 The Reasoning Framework (Week 1)

2.1 Chain-of-Thought (CoT) Reasoning

Standard LLMs map input prompts directly to output responses. In contrast, CoT reasoning introduces intermediate **thoughts**.

- **Concept:** Instead of asking the model for the answer directly ($Input \rightarrow Answer$), we prompt it to generate a reasoning path ($Input \rightarrow Reasoning \rightarrow Answer$).
- **Impact:** This improves performance on reasoning and arithmetictasks by decomposing complex problems into sub-problems.

2.2 Verifiers and Reward Models

The concept of **Verifier** was introduced in week 1. In the context of DeepSeek R1 and similar architectures, generating a thought is one part of whole process and evaluating it is another.

The Verifier acts as a **Reward Model** in the RL framework. It scores the reasoning steps, providing the feedback signal necessary for the model to learn. We reviewed two types of feedback:

1. **Outcome Supervision:** Rewarding the model only if the final answer is correct.
2. **Process Supervision:** Rewarding each step of the reasoning chain (Process Reward Models). This is generally more effective for complex reasoning tasks.

3 Reinforcement Learning Foundations

3.1 The Agent-Environment Interface

We defined the RL loop in which the **Agent** (our LLM) interacts with an **Environment** (the context/prompt) at discrete time steps t.

- **State (S_t):** The current situation (e.g., the text generated so far).
- **Action (A_t):** The decision made by the agent (e.g., predicting the next token).
- **Reward (R_{t+1}):** A scalar feedback signal indicating how good the action was.

The goal of the agent is to maximize the expected cumulative reward, denoted as the return G_t:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{1}$$

where γ is the discount factor ($0 \leq \gamma \leq 1$).

3.2 The Multi-Armed Bandit Problem

- **The Problem:** The agent must choose between k options (arms). The agent does not know the probability distribution of rewards for the arms.
- **Exploration vs. Exploitation:**
  - *Exploitation:* Choosing the arm with the highest estimated value to maximize immediate reward.
  - *Exploration:* Choosing a sub-optimal arm to gather data, possibly discovering a better arm.
- **ϵ-Greedy Algorithm:** In this algorithm, the agent explores with probability ϵ and exploits with probability $1 - \epsilon$.

4 Markov Decision Processes (MDPs)

4.1 Formal Definition

We formalized sequential decision-making using Markov Decision Processes. An MDP is defined by the tuple (S, A, P, R, γ). The core assumption is the **Markov Property**:

$$P(S_{t+1}|S_t, A_t) = P(S_{t+1}|S_t, A_t, S_{t-1}, A_{t-1}, \dots) \tag{2}$$

This implies that the current state contains all necessary information about the history to predict the future.

5 Week 2: Solving the MDP

In Week 2, we focused on the algorithms used to solve MDPs.

5.1 Value Functions

To choose the best action, we must estimate the "value" of being in a state.

- **State-Value Function $V_\pi(s)$:** How good is it to be in state s?
- **Action-Value Function $Q_\pi(s, a)$:** How good is it to take action a in state s?

The relationship between these values is defined by the **Bellman Expectation Equation**:

$$V_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V_\pi(s')] \tag{3}$$

This recursive equation allows us to compute values using Dynamic Programming if the model dynamics (p) are known.

5.2 Monte Carlo (MC) Methods

In the case of LLMs, we don't know the full transition dynamics of the environment. Monte Carlo methods solve this by learning from **experience**.

- **Mechanism:** The agent plays out full episodes. At the end of an episode, it calculates the return G_t and updates the value estimate $V(S_t)$ towards G_t.
- **Limitation:** MC methods can only update after an episode terminates.

5.3 Temporal Difference (TD) Learning

TD Learning is a central concept for our project. It combines the benefits of Dynamic Programming (bootstrapping) and Monte Carlo (sampling).

- **The Update Rule (TD(0)):**

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \tag{4}$$

- **Significance:** The agent updates its estimate based on an estimate of the next state ($V(S_{t+1})$). This allows the model to learn *during* the episode, step-by-step.
- **Application to LLMs:** This is crucial for training reasoning models, as we want the model to realize it made a logical error immediately, rather than waiting until the entire paragraph is generated.

6 Architecture Deep Dive: Transformers

(Based on supplementary resources shared on Transformer Architectures)

Reviewed the underlying architecture of LLMs: The Transformer.

6.1 Attention Mechanism

The core innovation of the Transformer is the Self-Attention mechanism, which allows the model to weigh the importance of different words in a sequence relative to each other.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{5}$$

Understanding this flow of information is vital when we later implement RL, as the policy $\pi(a|s)$ is parameterized by these attention weights.

7 Conclusion

1. Reasoning can be modeled as a sequential decision process (MDP).
2. Verifiers act as the source of truth (Reward) in this process.
3. TD Learning allows for efficient updates during the reasoning generation.